

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0702

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards (BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

I **NEELAKANDAMOORTHY C(192421063)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: NEELAKANDAMOORTHY C

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.
3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma_log.txt`.
4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class `StarSwitch` with methods `add_port(host_id)`, `remove_port(host_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

Question 1: Star Topology Diagram Generator

Question

Write a Python script that generates a **star topology diagram** for a given number of hosts and a central switch. It should display the host labels, switch name, and Cat6 UTP cable length.

Theory

What is a Network Topology?

A **network topology** describes how computers, networking devices, and communication links are arranged in a network.

Common topologies include:

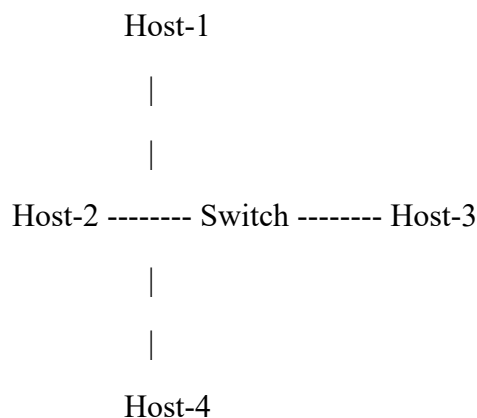
- Bus topology
- Star topology
- Ring topology
- Mesh topology
- Tree topology
- Hybrid topology

In this question, we are using **Star Topology**.

What is Star Topology?

In a **star topology**, all devices are connected to one **central networking device**, usually a switch.

Example:



The switch is the **center/hub of the network**.

Important Characteristics

1. Every host has a separate connection to the switch.
2. Hosts do not directly connect to each other.
3. The switch manages communication between hosts.
4. Adding a new host is easy.
5. If one cable fails, normally only that host is disconnected.
6. If the central switch fails, communication through the switch is affected.

What is Cat6 UTP?

Cat6 means Category 6 cable.

UTP means:

Unshielded Twisted Pair

It is commonly used for Ethernet networking.

The assessment requires the cable length to remain within the **100 m segment limit**.

Program Logic

The program performs these steps:

Start



Get number of hosts



Validate number of hosts



Create central switch



Create Host-1, Host-2, ...



Generate cable length



Check cable length $\leq$ 100 m



Print star topology

↓

Print summary

↓

Return host-length dictionary

↓

End

Why use a Dictionary?

We can store:

Host-1 → 82

Host-2 → 15

Host-3 → 4

Host-4 → 36

So a Python dictionary is suitable:

```
{  
    "Host-1": 82,  
    "Host-2": 15,  
    "Host-3": 4  
}
```

Complexity

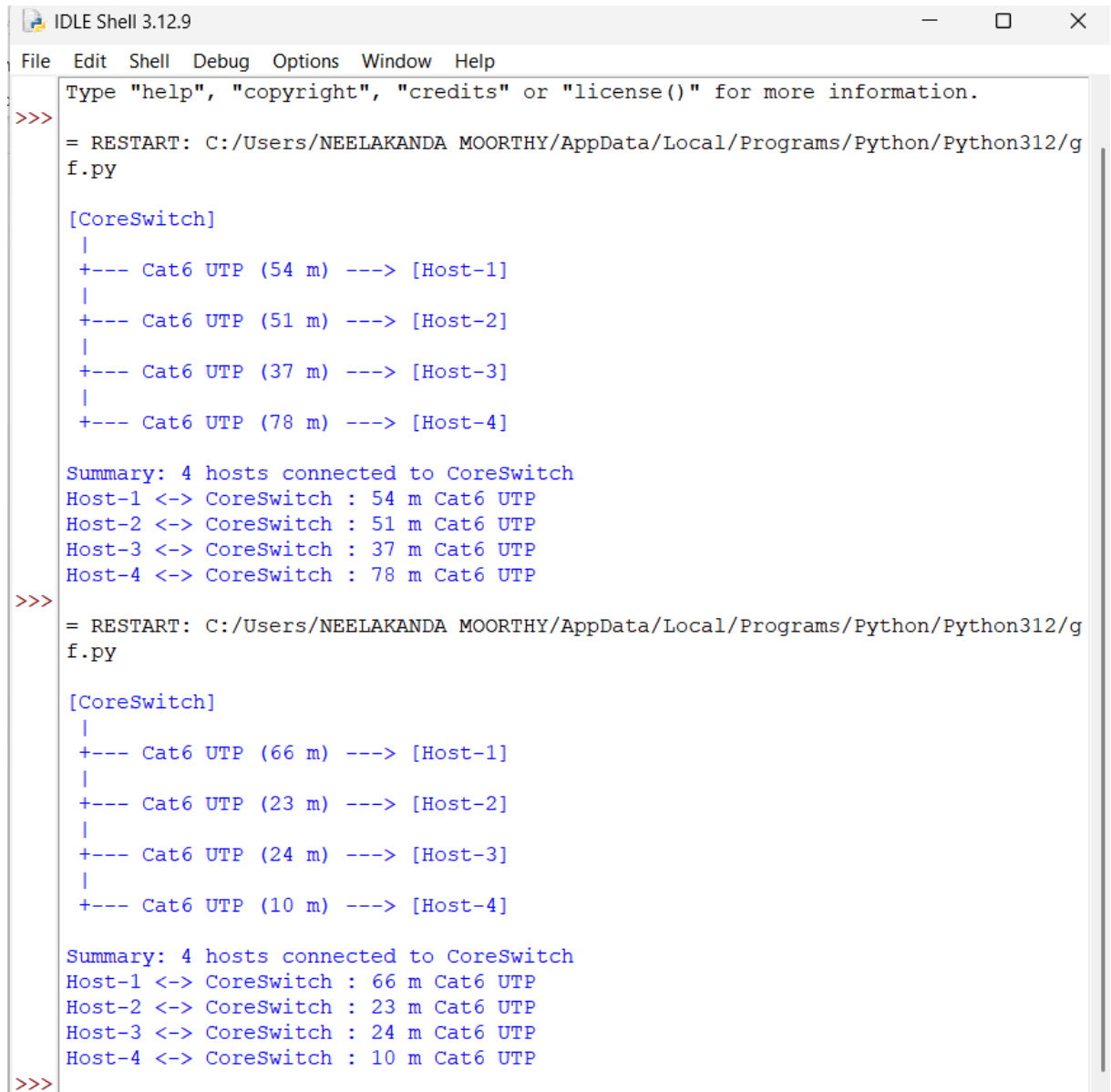
Time Complexity: $O(n)$

Because the program processes each host.

Space Complexity: $O(n)$

Because the dictionary stores one entry for each host.

7. PYTHON Code Screenshot



```
IDLE Shell 3.12.9
File Edit Shell Debug Options Window Help
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/g
f.py

[CoreSwitch]
|
+---- Cat6 UTP (54 m) ----> [Host-1]
|
+---- Cat6 UTP (51 m) ----> [Host-2]
|
+---- Cat6 UTP (37 m) ----> [Host-3]
|
+---- Cat6 UTP (78 m) ----> [Host-4]

Summary: 4 hosts connected to CoreSwitch
Host-1 <-> CoreSwitch : 54 m Cat6 UTP
Host-2 <-> CoreSwitch : 51 m Cat6 UTP
Host-3 <-> CoreSwitch : 37 m Cat6 UTP
Host-4 <-> CoreSwitch : 78 m Cat6 UTP
>>>
= RESTART: C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/g
f.py

[CoreSwitch]
|
+---- Cat6 UTP (66 m) ----> [Host-1]
|
+---- Cat6 UTP (23 m) ----> [Host-2]
|
+---- Cat6 UTP (24 m) ----> [Host-3]
|
+---- Cat6 UTP (10 m) ----> [Host-4]

Summary: 4 hosts connected to CoreSwitch
Host-1 <-> CoreSwitch : 66 m Cat6 UTP
Host-2 <-> CoreSwitch : 23 m Cat6 UTP
Host-3 <-> CoreSwitch : 24 m Cat6 UTP
Host-4 <-> CoreSwitch : 10 m Cat6 UTP
>>>
```

8.

Question 2: UTP Segment Validator (IEEE 802.3)

Question

Create a function:

`validate_segment(length_m)`

It should return True when the UTP cable length is within the ≤ 100 m limit and False with an error message otherwise. At least three test cases are required.

Theory

What is a Cable Segment?

A **cable segment** is a section of network cable connecting network devices.

For this question, we are checking the length of a UTP cable.

Why Validate Cable Length?

A network cable has a maximum supported segment length. The assessment specifically uses:

Maximum = 100 m

Therefore, the program must prevent an invalid cable length from being accepted.

Valid Values

Examples:

0 m → Valid

50 m → Valid

100 m → Valid

101 m → Invalid

150 m → Invalid

The basic condition is:

$0 \leq \text{length_m} \leq 100$

Function

A function is a reusable block of code.

Here:

```
def validate_segment(length_m):
```

`length_m` represents the cable length in metres.

Return Value

If the length is valid:

return True

If it is invalid:

return False

and an error message can be displayed.

Test Cases

We should test different situations:

Test Case 1

Input: 50

Output: True

Test Case 2

Input: 100

Output: True

Test Case 3

Input: 120

Output: False

Program Flow

Input cable length

↓

Is length between 0 and 100?

↓

Yes No

↓

↓

True Error

↓

False

Complexity

Time Complexity: $O(1)$

Only one condition is checked.

7. PYTHON Code S

```
gf.py - C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/gf...
File Edit Format Run Options Window Help
import random

def generate_star_topology(n_hosts, switch="CoreSwitch", min_len=1, max_len=100)
    if n_hosts <= 0 or min_len < 0 or max_len > 100 or min_len > max_len:
        raise ValueError("Invalid input")

    hosts = {"Host-{}".format(i): random.randint(min_len, max_len)
             for i in range(1, n_hosts + 1)}

    print(f"\n[{switch}]")
    for host, length in hosts.items():
        print(" |")
        print(f" +--- Cat6 UTP ({length} m) ---> [{host}]")

    print(f"\nSummary: {n_hosts} hosts connected to {switch}")
    for host, length in hosts.items():
        print(f"{host} <-> {switch} : {length} m Cat6 UTP")

    return hosts

generate_star_topology(4)
```

OUTPUT:

```
>> = RESTART: C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/gf.py
True
True
Error: Length > 100m
False
>>
```

Ln: 45 Col: 0

Question 3: CSMA/CD Simulation with Collision Logging

Question

Write a Python program that simulates **CSMA/CD on a star topology with 4 hosts**. The program should log collision events and back-off intervals to:

csma_log.txt

This is directly required by the assessment.

Theory

What is CSMA/CD?

CSMA/CD stands for:

Carrier Sense Multiple Access with Collision Detection

It is a method for managing communication when multiple devices share a communication medium.

Meaning of Each Part

Carrier Sense

A host first checks whether the communication medium is available.

Is the channel free?

Multiple Access

Multiple hosts can attempt to access the communication medium.

Collision Detection

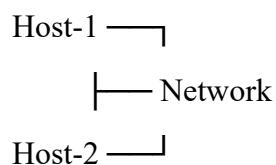
If two hosts transmit at the same time, a collision can occur.

The system detects the collision.

What is a Collision?

A **collision** happens when two or more devices transmit data at approximately the same time on a shared communication medium.

Example:



Both transmit



Collision

What is Back-off?

After detecting a collision, a host waits for a random amount of time before trying again.

This waiting period is called the **back-off interval**.

Example:

Host-1 → Collision → Wait 5 ms → Retry

Host-2 → Collision → Wait 2 ms → Retry

Random waiting helps reduce the chance of another immediate collision.

Four Hosts

The question specifically requires four hosts:

Host-1

Host-2

Host-3

Host-4

They can attempt transmissions.

Log File

The program must save information in:

csma_log.txt

For example:

Collision detected!

Host-1 back-off: 5 ms

Host-3 back-off: 2 ms

Why Logging is Important?

Logging allows us to:

- record collisions,
- record back-off times,

- examine network behavior,
- debug the simulation,
- verify that the program works.

Program Flow

Start



Create 4 hosts



Select transmitting hosts



Attempt transmission



Collision?



Yes —→ Record collision



Generate back-off time



Record back-off



Retry

Complexity

For n simulation iterations:

Time Complexity: $O(n)$

Space Complexity: $O(1)$ apart from the log file

7. VS Code Screenshot

```
kk.py - C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/kk....  
File Edit Format Run Options Window Help  
import random  
  
hosts = ["H1", "H2", "H3", "H4"]  
  
with open("csma_log.txt", "w") as f:  
    for i in range(5):  
        h1, h2 = random.sample(hosts, 2)  
        f.write(f"{h1}, {h2}: Collision!\n")  
        f.write(f"Backoff: {random.randint(1,10)} ms\n")  
  
print("Log saved in csma_log.txt")
```

OUTPUT:

```
>> |  
    = RESTART: C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/k  
    k.py  
    Log saved in csma_log.txt  
>> |
```

Ln 48 Col 0

Question 4: Socket-Based Star Switch Client-Server

Question

Using Python's socket library, create a client-server program that models two hosts communicating through a star switch. The switch's MAC address table should be printed after frame transmission.

Theory

What is a Socket?

A **socket** is an endpoint used for communication between two programs over a network.

Python provides the socket library for network communication.

```
import socket
```

Client

The **client** initiates communication.

For example:

Host-1 → Client

It connects to the server and sends a message.

Server

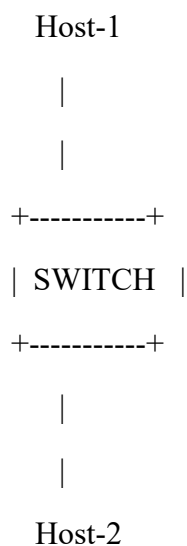
The **server** waits for incoming connections.

In our model:

Server → represents the central switch

Star Topology Model

The question can be represented as:



What is a MAC Address?

MAC stands for:

Media Access Control

A MAC address is used to identify a network interface at the data-link level.

Example:

AA:BB:CC:DD:01

AA:BB:CC:DD:02

MAC Address Table

A switch maintains information that associates devices with their network addresses/ports.

Example:

Host-1 → AA:BB:CC:DD:01

Host-2 → AA:BB:CC:DD:02

This is represented in Python using a dictionary:

```
mac_table = {  
    "Host-1": "AA:BB:CC:DD:01",  
    "Host-2": "AA:BB:CC:DD:02"  
}
```

What is a Frame?

At the data-link layer, data is transmitted in the form of **frames**.

Example:

Host-1

↓

Frame

↓

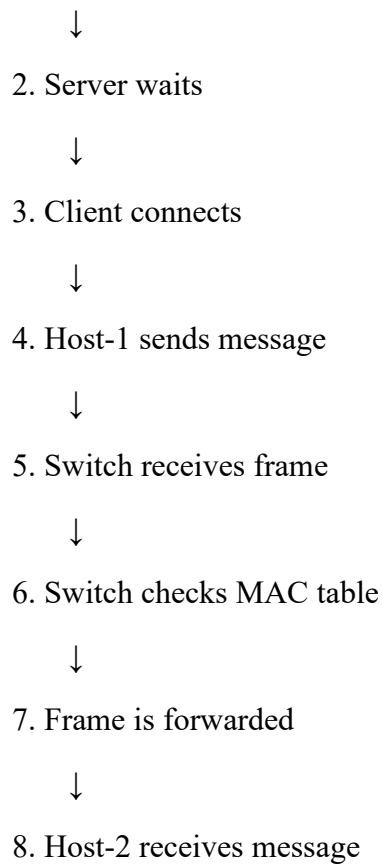
Switch

↓

Host-2

Communication Process

1. Start server



Important Socket Functions

socket()

Creates a socket.

```
socket.socket()
```

bind()

Associates the server with an address and port.

```
server.bind(("localhost", 5000))
```

listen()

Makes the server wait for connections.

```
server.listen()
```

accept()

Accepts a client connection.

```
conn, addr = server.accept()
```

connect()

Used by the client to connect to the server.

```
client.connect(("localhost", 5000))
```

send()

Sends data.

```
client.send(message.encode())
```

recv()

Receives data.

```
client.recv(1024)
```

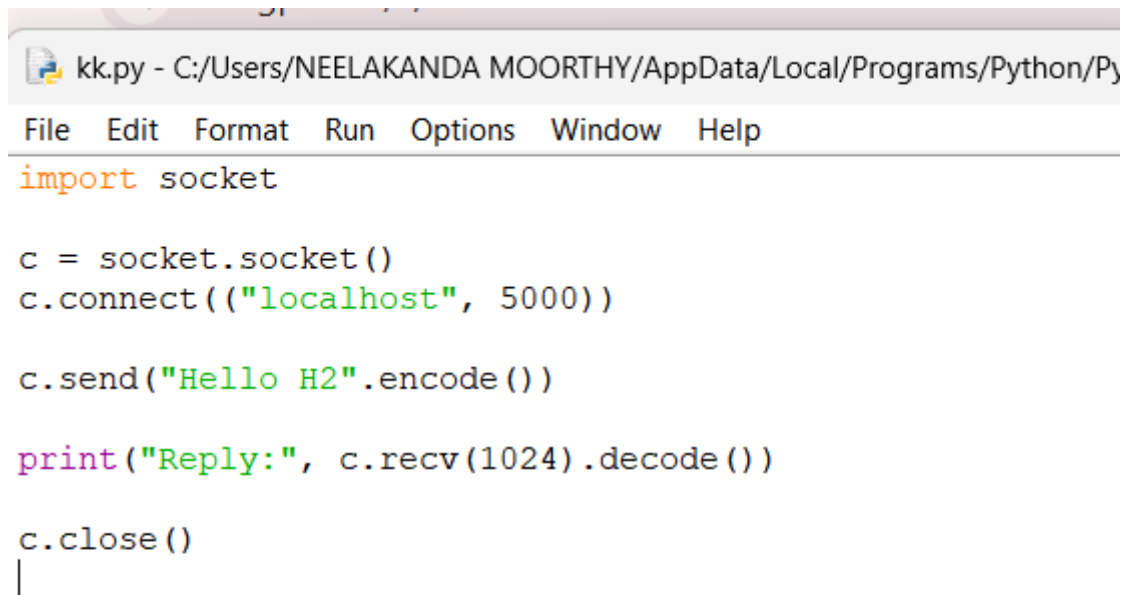
Complexity

For this simple two-host model:

Time Complexity: $O(1)$

Space Complexity: $O(1)$

7. VS Code Screenshot



```
kk.py - C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Py
File Edit Format Run Options Window Help
import socket

c = socket.socket()
c.connect(("localhost", 5000))

c.send("Hello H2".encode())

print("Reply:", c.recv(1024).decode())

c.close()
|
```

Question 5: StarSwitch Class

Question

Create a class named:

StarSwitch

with these methods:

add_port(host_id)

remove_port(host_id)

broadcast(frame)

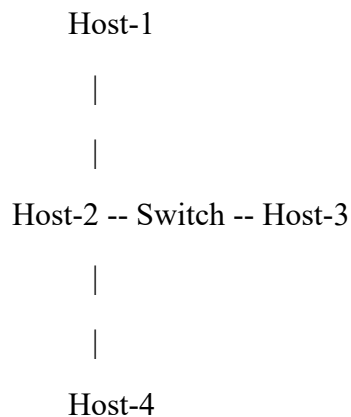
The broadcast must send the frame to all ports except the source and log each delivery.

Theory

What is a Switch?

A **network switch** is a networking device used to connect devices within a network.

In star topology, the switch is placed at the center.



What is a Class?

A Python **class** is a blueprint for creating objects.

Example:

```
class StarSwitch:
```

We create an object:

```
switch = StarSwitch()
```

What is a Port?

A switch has multiple ports through which hosts can connect.

For example:

Port 1 → Host-1

Port 2 → Host-2

Port 3 → Host-3

Port 4 → Host-4

add_port()

This method adds a host to the switch.

```
switch.add_port("Host-1")
```

After adding:

Host-1

is connected to the switch.

remove_port()

This method removes a host from the switch.

Example:

```
switch.remove_port("Host-4")
```

Now Host-4 is disconnected from the switch.

broadcast()

Broadcast means sending a frame to multiple connected ports.

Suppose:

Source = Host-1

The switch sends the frame to:

Host-2

Host-3

Host-4

It does **not** send it back to Host-1.

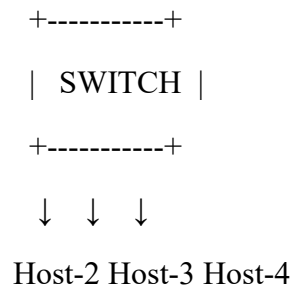
Broadcast Diagram

Host-1

SOURCE

|

↓



Frame

A frame can contain:

```
frame = {  
    "source": "Host-1",  
    "message": "Hello all hosts"  
}
```

The switch reads the source:

```
source = frame["source"]
```

Then loops through all connected ports:

```
for host in self.ports:
```

```
    and skips the source:
```

```
    if host != source:
```

Program Flow

Create StarSwitch



Add Host-1



Add Host-2



Add Host-3



Add Host-4



Create frame

↓

Broadcast frame

↓

Skip source

↓

Deliver to remaining hosts

↓

Log delivery

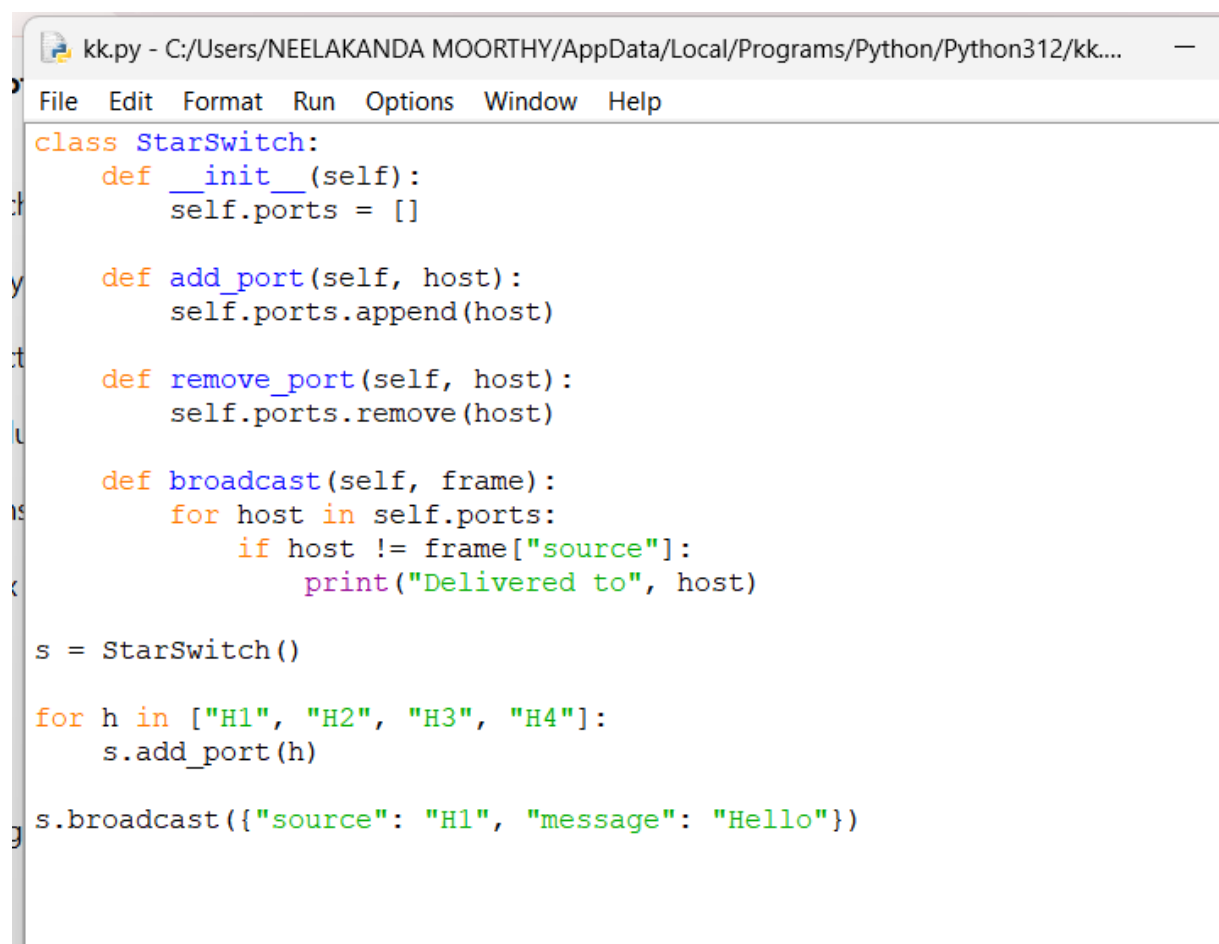
Complexity

If there are n ports:

add_port() → O(1)

remove_port() → O(n)

7. VS Code Screenshot



```
kk.py - C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/kk...
File Edit Format Run Options Window Help

class StarSwitch:
    def __init__(self):
        self.ports = []

    def add_port(self, host):
        self.ports.append(host)

    def remove_port(self, host):
        self.ports.remove(host)

    def broadcast(self, frame):
        for host in self.ports:
            if host != frame["source"]:
                print("Delivered to", host)

s = StarSwitch()

for h in ["H1", "H2", "H3", "H4"]:
    s.add_port(h)

s.broadcast({"source": "H1", "message": "Hello"})
```

OUTPUT:

```
= RESTART: C:/Users/NEELAKANDA MOORTHY/AppData/Local/Programs/Python/Python312/k  
k.py  
Delivered to H2  
Delivered to H3  
Delivered to H4  
> |
```